Ders 5 Genişletilmiş Notu: Açık Adresleme Analizi ve İleri Hashing Yöntemleri

Ders 5 Genişletilmiş Notu: Açık Adresleme Analizi ve İleri Hashing Yöntemleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Açık Adreslemenin Tekrarı
- 2. Başarısız Aramada Beklenen Deneme Sayısı
- 3. Yeniden Hashing (Rehashing)
- 4. Evrensel Hashing (Universal Hashing)
- 5. Evrensel Aile Kurmak: Asal Sayı Yöntemi
- 6. Evrensel Aile Kurmak: Matris Yöntemi
- 7. n Anahtarda En Az Bir Çakışma Olasılığı
- 8. Mükemmel Hashing (Perfect Hashing)
- 9. Mükemmel Hashingin İki Teoremi
- 10. Cuckoo Hashing
- 11. Hopscotch Hashing
- 12. Genişletilebilir Hashing (Extendible Hashing)
- Örnek Sorular

1. Açık Adreslemenin Tekrarı

Hoca derse geçen haftanın son konusu olan açık adreslemeyi (open addressing) hatırlatarak başladı, çünkü dersin ilk kısmı onun analiziydi. Hash tablosu kurarken karşılaşılan en büyük sorun çakışmalardır (collision). Bunları çözmenin iki yolu görülmüştü:

- **Ayrı zincirleme (separate chaining):** Bir slota denk gelen başka bir eleman varsa, o slotu gösteren bir bağlı liste oluşturulur ve çakışan elemanlar orada saklanır. Analizi geçen hafta yapılmıştı.
- **Açık adresleme:** Hash fonksiyonu, anahtara ek olarak ikinci bir parametre alır: kaçıncı denemede (probe) olunduğunu gösteren i. İlk denemede hesaplanan slot doluysa i artırılır ve i kullanılarak üretilen ikinci slot denenir. Böylece bütün elemanlar tek bir tabloda tutulur.

Açık adreslemenin deneme yöntemleri doğrusal deneme (linear probing), karesel deneme (quadratic probing) ve çift hashingdi (double hashing). Hepsinin ortak noktası, tablo boyutu m, eleman sayısı n iken tablonun eleman sayısından büyük tasarlanmasıdır. Yük faktörü (load factor) doluluk oranıdır: $\alpha = n/m$. Açık adresleme yapabilmek için slot sayısının daha fazla olması gerektiğinden burada $\alpha < 1$'dir.

2. Başarısız Aramada Beklenen Deneme Sayısı

2.1 Teorem

Slayttaki teorem şudur: yük faktörü $\alpha = n/m < 1$ olan bir açık adresli hash tablosunda, düzgün hashing (uniform hashing) varsayılırsa, başarısız bir aramadaki beklenen deneme sayısı

en fazla $1/(1-\alpha)$'dır (ekrandan, 00:10:00). Hoca slayttaki ifadede bir eşitlik işaretinin yanlış yazıldığını söyleyip düzeltti; ispatı Microsoft Whiteboard üzerinde yaptı.

2.2 İspatın Fikri

Her başarısız deneme, yani dolu (occupied) bir slota denk gelen her deneme, deneme sayısını bir artırır. Bu yüzden ardışık denemelerin dolu slota gelme olasılıklarına bakılır.

- Tablodaki dolu slot sayısı n, toplam slot sayısı m'dir. İlk denemenin dolu bir slota denk gelme olasılığı n/m'dir ("with the probability n/m first probe hits an occupied slot", ekrandan, 00:10:00). Bu durumda ikinci denemeye gerek vardır.
- İkinci denemede elimdeki elemanlardan biri ve denenecek slotlardan biri azaldığı için dolu slota gelme olasılığı $(n-1)/(m-1)$ olur.
- Üçüncü deneme için $(n-2)/(m-2)$ olur ve bu böyle sürer.

2.3 Rastgele Değişken ve Olaylar

X, başarısız bir aramada yapılan deneme sayısını gösteren rastgele değişkendir ("number of probes made in an unsuccessful search"). A_i ($i = 1, 2, ...$) ise i. denemenin yapılması ve dolu bir slota denk gelmesi olayıdır.

En az i deneme yapılması için ilk denemenin dolu slota gelmesi, ikincisinin de, üçüncüsünün de... dolu slota gelmesi gerekir; bu olayların hepsi arka arkaya gerçekleşmelidir. Dolu slotla her karşılaşmada bir deneme daha gerektiği için kesişim A_{i-1}'e kadar gider:

$$\Pr\{X \geq i\} = \Pr\{A_1 \cap A_2 \cap \cdots \cap A_{i-1}\} = \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \Pr\{A_{i-1} \mid A_1 \cap \cdots \cap A_{i-2}\}$$

Koşullu olasılıkların değerleri yukarıda bulunanlardır (ekrandan, 00:20:00):

$$\Pr\{X \geq i\} = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdots \frac{n-i+2}{m-i+2}$$

Hoca tahtada bir indisi önce yanlış yazdı ($i-1$ yerine $i-2$ olması gerekiyordu) ve düzeltti.

2.4 Üst Sınır

Bu çarpanlar genelleştirilebilir: $m > n$ olduğu bilindiği için

$$\frac{n-j}{m-j} \leq \frac{n}{m}$$

olur. Çarpımda $i-1$ terim olduğundan her terim yerine n/m yazılırsa üst sınır bulunur:

$$\Pr\{X \geq i\} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}$$

Kanıtlanmak istenen şey beklenen deneme sayısının $1/(1-\alpha)$ olduğuydu; bunun için i'den kurtulmak, i'yi sonsuza götürerek toplamak gerekir. Negatif olmayan tam sayı değerli bir rastgele değişkenin beklenen değeri $E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\}$ diye yazılır:

$$E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \sum_{i=0}^{\infty} \alpha^{i} = \frac{1}{1-\alpha}$$

Son eşitlik, i'yi 1 yerine 0'dan başlatınca elde edilen geometrik serinin toplam formülüdür. Böylece başarısız aramadaki beklenen deneme sayısının en fazla $1/(1-\alpha)$ olduğu ispatlanmış olur.

> **Ek bilgi (derste anlatılmadı):** $E[X] = \sum_{i \geq 1} \Pr\{X \geq i\}$ eşitliği, X yalnız $0, 1, 2, \ldots$ değerlerini alıyorsa geçerlidir. $|\alpha| < 1$ için $\sum_{i=0}^{\infty} \alpha^i = 1/(1-\alpha)$ geometrik seri formülüdür.

2.5 Sonucun Yorumu

- $\alpha = n/m = 0{,}5$ ise tablo yüzde elli doludur. Başarısız arama için en fazla $1/0{,}5 = 2$ deneme yapılır.
- $\alpha = 0{,}9$ ise tablo yüzde doksan doludur ve deneme sayısı $1/0{,}1 = 10$'a çıkar.

Yani açık adreslemede doluluk arttıkça deneme sayısı artar, işlemler çok zaman almaya başlar ve bir noktadan sonra başka bir çözüme gitmek gerekir.

3. Yeniden Hashing (Rehashing)

Tablo çok dolduğunda üretilen çözüm şudur: eskisinin iki katı büyüklüğünde yeni bir tablo oluşturulur, yeni bir hash fonksiyonu tanımlanır ve orijinal tablodaki silinmemiş bütün elemanların yeni hash değerleri hesaplanıp yeni tabloya aktarılır. Buna yeniden hashing (rehashing) denir. Pahalı bir işlemdir ama problemi çözer.

3.1 Örnek

Slayttaki örnekte (ekrandan, 00:23:40) tabloda 7 slot vardır ve $h(x) = x \bmod 7$'dir. 13, 15, 6 ve 24 doğrusal denemeyle yerleştirilmiştir:

Slot	0	1	2	3	4	5	6
İçerik	6	15		24			13

$13 \bmod 7 = 6$, $15 \bmod 7 = 1$, $24 \bmod 7 = 3$'tür. $6 \bmod 7 = 6$ dolu olduğu için doğrusal deneme başa sarar ve 6, 0. slota yerleşir (ara adım eklendi). Ardından 23 gelir: $23 \bmod 7 = 2$, slot 2'ye yerleşir. Tabloda artık 7 slotun 5'i doludur; tablo yüzde yetmişin üzerinde dolu olur ve yeni bir tablo oluşturulması gerekir.

Yeni tablo için eski boyutun iki katı olan 14 üzerinden bir asal sayı aranır ve 17 seçilir. Yeni fonksiyon $h(x) = x \bmod 17$ olur; eski değerlerin hepsinin yeni slotları bu fonksiyonla hesaplanır ve yerleştirilir. Böylece çakışma sorunu geçici bir süre için çözülmüş olur.

> **Ders sonrası not:** Hoca "14'e yakın asal sayı arıyorum ve 17'yi seçiyorum" dedi. 14'e en yakın asal 13'tür; seçilen 17, 14'ten büyük ilk asaldır. Kural, eski boyutun iki katından büyük ilk asalı seçmek diye okunmalıdır.

3.2 Avantaj ve Dezavantajlar

- Pahalı bir işlemdir: bütün elemanlar tek tek taranır; n eleman varken yeni tablo boyutu $2n$ civarına çıkar ve o kadar tarama yapılır.
- Ama sık yapılan bir işlem değildir; bu yüzden kullanışsız sayılmaz.
- Etkileşimli bir uygulamada talihsiz bir kullanıcı yeniden hashinge denk gelebilir: onun eklediği anahtar doluluk sınırını aştığı için yeniden hashing yapılır ve kullanıcı sistemde bir yavaşlama (slowdown) yaşar.

4. Evrensel Hashing (Universal Hashing)

4.1 Güvenlik Sorunu

Hashingle ilgili bir başka konu güvenliktir. Her hash fonksiyonu için çakışmaya sebep olacak bir anahtar serisi üretmek mümkündür; doğru anahtarlar verilirse her fonksiyon çakışmaya uğrar. Güvenlik kaygısı olan bir uygulamada, kötü niyetli bir kişinin en kötü durum senaryosunu bulamaması için farklı bir yöntem seçilmelidir.

Bunun için evrensel hashing yöntemi ortaya çıkmıştır. Fikri şudur: programın başında hash fonksiyonu, bir hash fonksiyonu ailesi içinden rastgele seçilir. Kötü niyetli kişi kodu ele geçirse bile hangi fonksiyonun kullanıldığını bilemeyeceği için yapıyı çökertecek diziyi oluşturamaz. Böylece bu tür uygulamaların sağlamlığı (robustness) artar.

4.2 Tanım

Slayttaki tanım (ekrandan, 00:27:35): H, verilen bir anahtar evreni U'yu $\{0, 1, \dots, m-1\}$ aralığına eşleyen sonlu bir hash fonksiyonu kümesi olsun. Farklı her $k, l \in U$ anahtar çifti için $h(k) = h(l)$ olan $h \in H$ fonksiyonlarının sayısı **en fazla** $|H|/m$ ise bu küme evrenseldir (universal).

Başka bir deyişle, H'den rastgele seçilen bir hash fonksiyonu için iki farklı anahtarın çakışma olasılığı $1/m$'den fazla değildir. Bu, k ve l $\{0, 1, \dots, m-1\}$ kümesinden rastgele ve bağımsız seçilseydi oluşacak çakışma olasılığıdır. Bu şartı sağlayan bir aile üretilebilirse evrensel hashing yapılabilir.

Hoca tanımı birden çok çifte de genişletti: $x_1 \neq y_1$, $x_2 \neq y_2$, ... iken $h(x_1) = h(y_1)$, $h(x_2) = h(y_2)$, ..., $h(x_k) = h(y_k)$ eşitliklerinin hepsini sağlayan fonksiyon sayısı en fazla $|H|/m^k$ olmalıdır. Gerekçesi: her çiftin çakışma olasılığı $1/m$'dir; k çiftin hepsinin üst üste gelmesi için $1/m$ k kez çarpılır ve bu ailedeki bütün fonksiyonlara yayılınca $|H|/m^k$ çıkar.

Evrensel bir aile oluşturmanın farklı yolları vardır. Derste ikisi anlatıldı: asal sayılarla ve matris yöntemiyle.

5. Evrensel Aile Kurmak: Asal Sayı Yöntemi

5.1 Kurallar

Slayttaki adımlar (ekrandan, 00:30:00):

- Yeterince büyük bir asal sayı p seçilir: olası her k anahtarı 0 ile $p-1$ aralığında kalmalıdır.
- Anahtar evreninin boyutu hash tablosundaki slot sayısından büyük kabul edildiği için $p > m$ olur.
- Evrensel aile, a ve b rastgele seçilerek şu fonksiyonlardan oluşur (ekrandan, 00:32:35):

$$H = \{h_{a,b}(x) = ((ax + b) \bmod p) \bmod m, \quad 1 \leq a \leq p-1,\ 0 \leq b \leq p-1\}$$

Ailede her (a, b) çifti için bir fonksiyon vardır; bu yüzden fonksiyonlar a, b indisiyle gösterilir. Örneğin rastgele 3 ve 7 gelirse fonksiyon $((3x + 7) \bmod p) \bmod m$, 4 ve 1 gelirse $((4x + 1) \bmod p) \bmod m$ olur. Her seferinde m'e göre mod alındığı için sonucun tablonun içinde bir yere denk gelmesi garanti altındadır. a için $p-1$, b için p seçenek olduğundan ailede $p(p-1)$ olası hash fonksiyonu vardır.

5.2 Örnek

$p = 17$, $m = 6$; altı slotluk bir tablo oluşturuluyor (ekrandan, 00:32:35):

- $h_{5,7}(3)$: $(5 \cdot 3 + 7) \bmod 17 = 22 \bmod 17 = 5$; $5 \bmod 6 = 5$.
- $h_{3,4}(28)$: $(3 \cdot 28 + 4) \bmod 17 = 88 \bmod 17 = 3$; $3 \bmod 6 = 3$.
- $h_{10,2}(3)$: $(10 \cdot 3 + 2) \bmod 17 = 32 \bmod 17 = 15$; $15 \bmod 6 = 3$. (Bu satır slaytta vardı, hoca sözlü olarak okumadı.)

Böylece farklı fonksiyonlarla farklı anahtarlar için farklı eşlemeler üretilir. Hesaplar python ile doğrulandı.

6. Evrensel Aile Kurmak: Matris Yöntemi

6.1 Kurallar

Hoca kuralların ilk bakışta karışık gelebileceğini, örnekle oturacağını söyledi.

- Anahtarın u bit uzunluğunda olduğu kabul edilir. u, anahtarların alabileceği en büyük ve en küçük değere göre belirlenir.
- m uzunluklu bir tablo oluşturulacaksa m ikinin tam kuvveti seçilir: $m = 2^b$.
- Rastgele bir $b \times u$ boyutlu ikili (0/1) matris h oluşturulur. Bu matris hash fonksiyonunun kendisidir.
- Anahtar x, bu matrisle çarpılarak eşlenir; toplama mod 2 ile yapılır: $h(x) = hx$. Slaytta bu matrislerin "kısa ve şişman" (short and fat) olduğu yazar.

6.2 Örnek

Dört bitlik bir anahtar ve 3×4 boyutlu bir hash matrisi (ekrandan, 00:35:10):

$$h = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad h(x) = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

Standart matris çarpımı yapılır, ama mod 2 ile:

- 1. satır: $1 \cdot 1 + 0 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 = 1$.
- 2. satır: $0 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 = 1$.
- 3. satır: $1 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 = 2$, mod 2 ile 0 (ara adım eklendi).

6.3 Ailede Kaç Fonksiyon Var?

Hoca öğrencilere $b \times u$ boyutlu matrislerle kaç hash fonksiyonu üretilebileceğini sordu. Cevap gelmeyince kendisi hesapladı: matriste $u \cdot b$ eleman vardır ve her eleman için iki seçenek vardır (0 ya da 1). Dolayısıyla 2^{ub} farklı fonksiyon üretilebilir.

Örnek: 32 bitlik IP adresleri için 256 slotluk bir tablo kurulmak isteniyor. $256 = 2^8$ olduğundan $b = 8$, $u = 32$'dir; 8×32 boyutlu bir 0/1 matris kullanılır ve $2^{8 \cdot 32}$ farklı hash fonksiyonu oluşturulabilir. Her seferinde matris programın başında rastgele seçildiği için kötü niyetli kullanıcı fonksiyona ulaşamaz.

6.4 Matris Yönteminin Evrensel Olduğunun İspatı

Bir ailenin evrensel olması için iki farklı anahtarın aynı hash değerini verme olasılığının $1/m$ olması gerekiyordu. Teorem: $x \neq y$ ise

$$\Pr[h(x) = h(y)] = \frac{1}{m} = \frac{1}{2^b}$$

(ekrandan, 00:45:30). Burada b matrisin satır sayısı, u anahtarın bit sayısıdır.

İspat tahtada aynı örnek üzerinden yapıldı:

- x ve y birbirinden farklıysa en az bir bitte mutlaka farklıdırlar. Hoca en kötü durum senaryosu olarak yalnız bir bitte, son bitte farklı olduklarını varsaydı: tahtada $x = (1, 0, 1, 0)$, $y = (1, 0, 1, 1)$. Bu bit i. bit olsun: $x_i = 0$, $y_i = 1$.
- Çarpımda bu bite denk gelen sütun matrisin i. sütunudur (h_i). Matrisin bu sütun çıkarılmış hâline h' denir.
- $x_i = 0$ olduğu için i. sütun ne olursa olsun çarpım onu sıfırlar; dolayısıyla $h(x) = h'(x)$.
- $y_i = 1$ olduğu için i. sütundaki değerler toplama eklenir: $h(y) = h'(y) + h_i$.
- Diğer bitler aynı kabul edildiği için $h'(x) = h'(y)$'dir. O hâlde $h(x)$ ile $h(y)$'yi farklı kılabilecek tek şey h_i'dir.
- h_i sütununda b eleman vardır, yani 2^b seçenek vardır. Bunlardan yalnız biri (hepsi sıfır olan sütun) iki değeri eşit yapar. Sonuç $1/2^b = 1/m$'dir.

Hoca ayrıca "daha fazla bit farklı olsaydı çakışma olasılığı düşerdi; ne kadar örtüşürse çakışma o kadar artar" dedi.

Ders sonrası not: Hoca, bitler daha çok farklıysa çakışma olasılığının düşeceğini söyledi; doğrusu, $x \neq y$ olan her durumda olasılık tam $1/2^b$'dir. Ayrımın yapıldığı i. sütun dışındaki sütunlar sabitlendiğinde $h(x) = h(y)$ olması için h_i'nin tek bir belirli vektöre, $h'(x) + h'(y)$ (mod 2) vektörüne eşit olması gerekir. h_i 2^b değerden birini eşit olasılıkla aldığı için olasılık kaç bit farklı olursa olsun $1/2^b$'dir. Hocanın tek bit varsayımı ispatı sadeleştirir ama sonucu değiştirmez.

7. n Anahtarda En Az Bir Çakışma Olasılığı

7.1 Soru

Aradan sonra hoca şu soruyu sordu: rastgele seçilmiş bir hash fonksiyonu için n anahtar varsa, bunlar içinde en az bir çakışmayla karşılaşma olasılığı nedir? Bu olasılık kullanım sırasında performansı etkiler ve parametreler buna göre seçilir.

7.2 Hesap

Önce kaç farklı anahtar çifti olduğu bulunur (ekrandan, 01:06:45). Sıranın önemli olmadığı şekilde n anahtardan iki anahtar seçmek kombinasyondur:

$$\binom{n}{2} = \frac{n!}{(n-2)!\,2!} = \frac{n(n-1)(n-2)\cdots 1}{(n-2)(n-3)\cdots 1 \cdot 2} = \frac{n(n-1)}{2}$$

En az bir çakışma olasılığı, hiç çakışma olmaması olasılığının 1'den çıkarılmasıyla bulunur:

$$\Pr(\#\text{çakışma} \geq 1) = 1 - \Pr(\#\text{çakışma} = 0)$$

Bir çiftin çakışma olasılığı slot sayısıyla orantılıdır: $1/m$. Bunu 1'den çıkarınca bir çiftin çakışmama olasılığı $1 - 1/m$ bulunur. Hiç çakışma olmaması için hiçbir çiftin çakışmaması gerekir; çift sayısı $n(n-1)/2$ olduğu için hoca bu olasılığı çiftler bağımsızmış gibi çarparak yazdı:

$$\Pr(\#\text{çakışma} \geq 1) = 1 - \left(1 - \frac{1}{m}\right)^{n(n-1)/2} = 1 - \left(\frac{m-1}{m}\right)^{n(n-1)/2}$$

Ek bilgi (derste anlatılmadı): Çiftlerin çakışma olayları birbirinden tam bağımsız değildir (aynı anahtar birden çok çiftte yer alır). Bu yüzden formül kesin değil, yaklaşık bir değerdir; doğum günü problemindeki yaklaşımla aynı mantıktadır.

7.3 Büyük Tabloda Evrensel Hashing

Slayttaki grafik (ekrandan, 01:10:00), $n = 1000$ anahtar için tablo boyutu m 1 milyona kadar değişirken bu olasılığı ($y = 1 - ((m-1)/m)^{n(n-1)/2}$) gösterir. Grafiğe bakılınca ancak çok büyük tablo boyutlarında kabul edilebilir çakışma oranları elde edildiği görülür. Olasılığın %50 civarına inmesi için tablonun 700 binler civarında olması gerekir.

- $n = 1000$, $m = 100000$ için $\Pr(\#\text{çakışma} \geq 1) = 0{,}9932$.
- 100 rastgele fonksiyon seçilirse hepsinde çakışma olma olasılığı $0{,}9932^{100} = 0{,}507$.
- Bunu 1'den çıkarınca, 100 fonksiyondan en az birinin çakışmasız olma şansı yaklaşık %50 bulunur.
- Hoca ayrıca 300 rastgele fonksiyon için hesap yaptı: $0{,}9932^{300} \approx 0{,}13$; en az birinin çakışmasız olma olasılığı yaklaşık %87.

Bu hesaplar yük faktörünün %1 olduğu durum içindir ($1000/100000$). Doluluk çok düşük olduğu hâlde çakışma oranları yüksektir. Büyük tablolar için bu kadar düşük yük faktörü elde etmek de tabloyu iyice büyütmeyi gerektirir. Slaytın sonucu: bu, mükemmel bir hash fonksiyonu seçmek için iyi bir yol değildir. Değerler python ile doğrulandı.

7.4 Ortalama $O(1)$'den En Kötü Durum $O(1)$'e

Şimdiye kadar görülen hash fonksiyonları, makul yük faktörleri ve uygun fonksiyonlarla insert, delete ve search işlemlerini **ortalama** $O(1)$'de yapabiliyordu. Bundan sonra en kötü durumda (worst case) bile $O(1)$ erişim veren yöntemlere bakıldı.

8. Mükemmel Hashing (Perfect Hashing)

8.1 Ne Zaman Kullanılır?

Mükemmel hashing, anahtar kümesinin değişmediği (statik olduğu) problemler için iyi bir çözümdür. Örnekler: bir CD-ROM'a veri yazmak ya da derleyici tasarımında ayrılmış kelimelere (reserved words) erişmek. Mükemmel hashing, daha iyisini bulacak şekilde hash fonksiyonunu sürekli deneyip gözden geçirme mantığıyla çalışır; gerekirse yeniden hashing yapılır. Sonuçta işlem $O(1)$'de çözülür.

8.2 Yapı

- İki seviyeli hashing yapılır ve her seviyede evrensel hashing kullanılır.
- Çakışan elemanlar için bağlı liste yerine küçük ikinci seviye hash tabloları vardır. Çakışma olan j slotu için yeni bir tablo ve ona ait bir h_j fonksiyonu tanımlanır.
- Birinci seviyede çakışma olabilir; ama ikinci seviyede çakışma yaratmayacak fonksiyonlar bulunmalıdır. Bunun için birkaç deneme yapılır.
- İkinci seviyede çakışma olasılığını elemek için j slotunun ikinci tablosunun boyutu m_j, o slotta çakışan anahtar sayısının karesi seçilir: $m_j = n_j^2$.

  Hocanın vurgusu: m_j'nin n_j^2 olması önemlidir.

Bu karesel ilişki ilk bakışta yer gereksinimini (storage requirement) çok artıracakmış gibi görünür. Ama birinci fonksiyon iyi tanımlanırsa ikinci seviyenin beklenen yer karmaşıklığı $O(n)$ düzeyinde tutulabilir (ispatı 9. bölümde).

Yol şöyledir: birinci hash fonksiyonu dikkatli seçilir. Her slot için ikinci fonksiyon h_j belirlenir; p değişmez, $m_j = n_j^2$ alınır ve a, b rastgele seçilerek ikinci fonksiyon üretilir. Sonra ikinci fonksiyonun çakışmaya sebep olmadığı kontrol edilir. Birinci (primary) fonksiyonun iyi olduğu da bu adımlarla kontrol edilir.

8.3 Hazır Bir Mükemmel Hash Tablosu

Hoca önce tamamlanmış bir tabloyu gösterdi. Anahtarlar 10, 22, 37, 40, 52, 60, 70, 72, 75'tir. Birincil hash için $((ak+b) \bmod p) \bmod m$ kullanılır; $a = 3$, $b = 42$, $p = 101$, $m = 9$ seçilmiştir. Evrensel hashingde anlatılan a, b, p kuralları burada da aynen geçerlidir.

- 10 için: $3 \cdot 10 + 42 = 72$, $72 \bmod 101 = 72$, $72 \bmod 9 = 0$; 10 sıfırıncı göze yerleşir. (Hoca önce 10'un ikinci göze yerleştiğini söyleyip sonra düzeltti.)
- 60, 72 ve 75 ikinci slotta çakışır. İkinci slotun fonksiyonu $m = 9$, $a = 10$, $b = 18$'dir ve 60, 72, 75'i sırasıyla 3, 4 ve 7 numaralı gözlere yerleştirir.
- Yedinci slotta da çakışma vardır; onlar için de ayrı ikinci fonksiyon oluşturulmuştur.

Hesaplar python ile doğrulandı: birincil fonksiyonla 10 → 0; 60, 72, 75 → 2; 70 → 5; 22, 37, 40, 52 → 7. İkinci fonksiyonla $(10 \cdot 60 + 18) \bmod 101 = 12$, $12 \bmod 9 = 3$; 72 → 4; 75 → 7.

8.4 Adım Adım Örnek

İkinci örnekte (ekrandan, 01:19:05 ve 01:20:00) adımlar tek tek kontrol edildi.

- $K = \{8, 22, 36, 75, 61, 13, 84, 58\}$, $n = 8$, $m = 8$.
- Fonksiyonlar: $h(k) = ((ak + b) \bmod p) \bmod m$ ve $h_j(k) = ((a_j k + b_j) \bmod p) \bmod m_j$.

Adım 1: a ve b rastgele seçilir, p anahtarlardan büyük seçilir: $p = 87$, $a = 64$, $b = 5$.

Adım 2: Her anahtarın ev hücresi $h(k)$ hesaplanır ve her slottaki eleman sayısı tutulur:

Slot	0	1	2	3	4	5	6	7
Anahtarlar	—	61	8, 84	—	75	22	13	36, 58
n_j	0	1	2	0	1	1	1	2

8 ile 84 ikinci slotta, 36 ile 58 yedinci slotta çakışır; diğer slotlarda birer eleman vardır.

Adım 3: $\sum_{j=0}^{m-1} n_j^2 < 2n$ kontrol edilir. Her slottaki eleman sayısının karesi toplanır:

$$\sum_{j=0}^{7} n_j^2 = 0 + 1 + 4 + 0 + 1 + 1 + 1 + 4 = 12 < 2 \cdot 8 = 16$$

Şart sağlanır; alt tablolara geçilebilir.

Adım 4: Alt tablolar yeni hash fonksiyonlarıyla doldurulur. Her alt tablonun m_j, a_j, b_j değerleri rastgele oluşturulmuştur:

Slot	m_j	a_j	b_j	Alt tablo
1	1	0	0	[61]
2	4	82	53	[84, 8, /, /]
4	1	0	0	[75]
5	1	0	0	[22]
6	1	0	0	[13]
7	4	10	54	[/, 58, 36, /]

Hoca işlemleri tek tek yapmadı, takip etmeyi öğrencilere bıraktı; yalnız 61'i gösterdi: $m = 1$, $a = b = 0$ olduğu için 0 mod p mod $m = 0$; 61 ikinci tablonun 0. gözüne yazılır. Önemli olan, ikinci hashlerin indekslerinin ve parametrelerinin ayrı olmasıdır. Bütün elemanlar yerleştirildiğinde ikinci seviyede çakışma olmadı; demek ki işlem düzgün yapıldı.

Python ile kontrol (ara adım eklendi): $(82 \cdot 84 + 53)$ mod 87 mod $4 = 0$, $(82 \cdot 8 + 53)$ mod 87 mod $4 = 1$; $(10 \cdot 58 + 54)$ mod 87 mod $4 = 1$, $(10 \cdot 36 + 54)$ mod 87 mod $4 = 2$. Tablodaki yerleşimle aynıdır.

9. Mükemmel Hashingin İki Teoremi

İşlemler iki kontrol üzerinden yürütülmüştü. Bu bölümde ikisinin ispatı yapıldı.

9.1 Toplar ve Kutular (Balls and Bins)

Hash fonksiyonları "toplar ve kutular" problemine benzer: değerleri slotlara koymak, topları kutulara yerleştirmek gibidir.

Teorem: n top, $m = n^2$ kutuya (rastgele) yerleştirilirse, hiçbir kutunun birden fazla top içermeme olasılığı en az $1/2$'dir. Bu, ikinci seviye tablo boyutunun neden n_j^2 seçildiğini gösteren teoremdir.

İspat: Yine çift sayısı üzerinden gidilir. Farklı çift sayısı $\binom{n}{2} = \frac{n(n-1)}{2}$'dir. İki farklı anahtarın aynı slota düşme olasılığı $1/m$'dir. Tüm tabloda beklenen çakışma sayısı, çift sayısı çarpı $1/m$'dir:

$$E[\#\text{çakışma}] = \binom{n}{2} \cdot \frac{1}{n^2} = \frac{n(n-1)}{2n^2} = \frac{n-1}{2n} < \frac{1}{2}$$

Tüm tabloda beklenen çakışma sayısı $1/2$'den küçük olduğuna göre en az bir çakışma olma olasılığı da $1/2$'den küçüktür. Böylece teorem ispatlanır.

Ek bilgi (derste anlatılmadı): "Beklenen çakışma sayısı $< 1/2$ ise en az bir çakışma olasılığı $< 1/2$" adımı Markov eşitsizliğidir: negatif olmayan Y için $\Pr(Y \geq 1) \leq E[Y]$.

9.2 İkinci Seviye Tabloların Toplam Boyutu

İkinci seviye tablolar kurulurken çok fazla yer kaybedileceğinden korkulmuştu. Teorem: $m = n$ slotlu birinci tabloya n anahtar yerleştirilirse, ikinci seviye tabloların beklenen toplam boyutu $2n$'den küçüktür:

$$E\left[\sum_{j=0}^{m-1} n_j^2\right] < 2n$$

(ekrandan, 01:30:00). Burada n_j, j. slota düşen anahtar sayısıdır; her slot için kurulan alt tablonun boyutu n_j^2 olduğundan bu toplam alt tabloların toplam boyutudur. Yani alt tablolar oluşturulduğunda tablo boyutu çok büyümez.

İspat (ekrandan, 01:36:25). Şu eşitlik kullanılır:

$$a^2 = a + 2\binom{a}{2}$$

Çünkü $\binom{a}{2} = \frac{a(a-1)}{2}$; bunu 2 ile çarpıp a eklenince a^2 çıkar. Bu eşitlik n_j^2'ye uygulanır:

$$E\left[\sum_{j=0}^{m-1} n_j^2\right] = E\left[\sum_{j=0}^{m-1} \left(n_j + 2\binom{n_j}{2}\right)\right]$$

Beklenen değerin doğrusallığıyla (linearity of expectation) ifade ikiye ayrılır; 2 dışarı çıkarılır:

$$= E\left[\sum_{j=0}^{m-1} n_j\right] + 2E\left[\sum_{j=0}^{m-1} \binom{n_j}{2}\right]$$

- İlk terim: bütün slotlardaki eleman sayıları toplanınca toplam eleman sayısı bulunur; $\sum n_j = n$.
- İkinci terimdeki $\binom{n_j}{2}$, j. slotta çakışan çift sayısıdır. Bütün slotlar için toplanınca tüm tablodaki çakışan çift sayısı bulunur. Bunun beklenen değeri çift sayısı çarpı çakışma olasılığıdır:

$$E\left[\sum_{j=0}^{m-1} \binom{n_j}{2}\right] = \binom{n}{2}\frac{1}{m} = \frac{n(n-1)}{2m} = \frac{n-1}{2} \quad (m = n)$$

Yerine konunca:

$$E\left[\sum_{j=0}^{m-1} n_j^2\right] = n + 2 \cdot \frac{n-1}{2} = 2n - 1 < 2n$$

İfade $2n-1$'e eşit olduğu için $2n$'den küçüktür ve ispat tamamlanır. Hoca ispatın biraz karışık olduğunu, not alınmasını istedi.

10. Cuckoo Hashing

10.1 Fikir

Cuckoo hashing, diğerlerine göre biraz daha güncel bir yöntemdir. Amacı işleri en kötü durumda $O(1)$ karmaşıklıkla yapabilmektir. Adını guguk kuşundan (cuckoo) alır: bu kuş yumurtalarını bırakmak için yuvalara bakar, orada başka yumurtalar varsa onları atıp kendi yumurtalarını koyarmış. Yöntem bu davranıştan esinlenmiştir.

İki farklı hash fonksiyonuyla çalışır ve haliyle iki hash tablosu oluşur. Her anahtar iki farklı fonksiyona verilir ve iki tablodan birinde yer edinebilir. Önce birinci tabloda denenir; yer bulunamazsa ikinci tabloda, anahtarın ikinci fonksiyon değerinin gösterdiği slota bakılır. Orası boşsa eleman oraya yerleşir. Boş değilse eleman yine oraya konur ve oradaki eski anahtar başka bir yere gönderilir; o da iki fonksiyonuna bakarak kendine yer arar. Bu, hepsi yerleşene kadar yinelemeli olarak yapılır.

10.2 İşlemler

Slayttaki tanımlar (ekrandan, 01:40:00):

- **Query(x):** $T[f(x)]$ ve $T[g(x)]$ kontrol edilir. Bakılacak topu topu iki yer vardır; sorgu oldukça basittir.
- **Delete(x):** x sorgulanır, bulunursa silinir.
- **Insert(x):** $T[f(x)]$ boşsa x oraya konur, iş biter.
- Değilse, $T[f(x)]$'te başlangıçta y olsun. x yine $T[f(x)]$'e konur ve y, $T[f(y)]$ ile $T[g(y)]$'den az önce atılmadığı yere gönderilir.
- O yer boşsa iş biter. Değilse y yine oraya konur ve süreç tekrarlanır: yeni atılan z, $T[f(z)]$ ve $T[g(z)]$'den şu anda y'yi içermeyen yere taşınır.
- Bu, ya iş bitene ya da $6 \log n$ elemanın atılması (bump) sınırına gelinene kadar sürer. $6 \log n$ eleman atıldıysa yeni bir f, g çifti seçilir ve tablodaki her eleman yeniden hashlenir.

Durma sınırı gereklidir: ya hepsi yerleşir ya da elemanlar sürekli yer değiştirip sonsuz döngüye girebilir. $6 \log n$'den fazla atma yapılıyorsa doluluk çok artmış demektir ve fonksiyonlar yeniden hashlenmelidir.

10.3 Karmaşıklık

Her an x'in yeri sabittir: ya T_1'de ya T_2'dedir. Bu yüzden sorgu ve silme $O(1)$'dir. Eklemenin zaman analizi daha zordur; hoca bunu derste çözmeyeceğini, beklendiği kadar kötü olmadığını söyledi.

10.4 Örnek

Hoca önce tamamlanmış bir cuckoo tablosu gösterdi: her anahtarın yanında iki değer vardır; soldaki birinci tabloya, sağdaki ikinci tabloya yerleşirse hangi göze gideceğini gösterir. Sonra aradan sonra tabloyu tahtada adım adım oluşturdu (ekrandan, 02:00:00). İki tablonun gözleri 0'dan 4'e numaralıdır. Hash değerleri (tablo 1, tablo 2):

Anahtar	A	B	C	D	E	F
Tablo 1	0	0	1	1	3	3
Tablo 2	2	0	4	0	2	4

Anahtarlar verilen sırayla yerleştirilir:

- A, tablo 1'in 0. gözüne yerleşir.
- B de tablo 1'in 0. gözüne yerleşmek ister. B'nin ikinci seçeneği boştur; ama cuckoo hashingin çalışma mantığı oraya bakmak değildir: eleman ilk tercihine yerleştirilir, oradaki başka yere gönderilir. B 0. göze konur, A ikinci seçeneğine, tablo 2'nin 2. gözüne gider.
- C, tablo 1'in 1. gözüne yerleşir.
- D de tablo 1'in 1. gözüne yerleşmek ister. D oraya konur, C ikinci tercihi olan tablo 2'nin 4. gözüne gönderilir.

- E, tablo 1'in 3. gözüne yerleşir.
- F de tablo 1'in 3. gözünü ister. F oraya konur; E tablo 2'nin 2. gözüne gider. Orada A vardır; A gönderilir ve tablo 1'in 0. gözüne gider. Orada B vardır; B gönderilir ve tablo 2'nin 0. gözüne yerleşir.

Sonuç:

Göz	0	1	2	3	4
Tablo 1	A	D		F	
Tablo 2	B		E		C

> **Hocanın vurgusu:** Cuckoo hashingde eleman ikinci seçeneği boş olsa bile önce ilk tercih ettiği yere konur ve oradaki eleman atılır.

Ardından yeni bir eleman eklendi: G'nin değerleri 1 ve 2'dir.

- G birinci tercihine, tablo 1'in 1. gözüne konur; D gönderilir ve tablo 2'nin 0. gözüne gider.
- B gönderilir, tablo 1'in 0. gözüne gider; A gönderilir, tablo 2'nin 2. gözüne gider.
- E gönderilir, tablo 1'in 3. gözüne gider; F gönderilir, tablo 2'nin 4. gözüne gider.
- C gönderilir, tablo 1'in 1. gözüne gelir ve G'yi atar. G yine elde kalmıştır.

Elde aynı eleman varken dönüp dolaşıp aynı slota geliniyorsa işin içinden çıkılamaz bir durumla karşılaşılmıştır. Bu kötü durum senaryosudur ve tabloların yeniden hashlenmesi gerektiğini gösterir. Python simülasyonu da aynı elemanların aynı slotlar arasında döngüye girdiğini doğruladı.

11. Hopscotch Hashing

11.1 Fikir

Bu yöntem seksek oyunundan (hopscotch) ilham alır. Yaptığı şey klasik doğrusal denemeyi geliştirmektir.

Doğrusal deneme kümelenmeye (clustering) yol açıyordu: bir eleman slotuna yerleştirilemezse hemen altındaki gözlere bakılıp ilk boş göze konuyordu. Bu da aranan elemanın kendi hash değerinin gösterdiği slotta değil, daha aşağılarda durmasına ve arama karmaşıklığının artmasına sebep oluyordu. Bunu çözmek için karesel deneme ve çift hashing görülmüştü.

Ama doğrusal denemenin bir avantajı da vardır: büyük veriyle çalışırken diskten belleğe bir anda çekilen veri içinde birbirine yakın elemanların bulunma ihtimali yüksektir. Bilgisayar donanımı düşünüldüğünde kullanışlı bir yöntemdir; biraz iyileştirilmesi gerekir.

İyileştirme şöyledir: çakışmada arka arkaya yerleşecek elemanların uzaklığına bir sınır konur. Bilgisayarın özelliklerine bakılarak bir en büyük uzunluk (komşuluk boyu, H) belirlenir; bir veriye ulaşıldığında hemen altındaki veriler de belleğe çekilmiş olur ve hızlı erişilir. Yerleştirmede önce doğrusal denemeyle elemanın nereye konabileceğine bakılır. Orası olması gereken yerden çok uzaksa, elemanı komşuluğuna, yani kendi orijinal hash değerine yaklaştıracak şekilde boşluk açılmaya çalışılır.

Boşluk açmak için yer değiştirecek (tahliye edilecek) elemanlar seçilir; bu rastgele değil, belirli bir algoritmayla deterministik olarak yapılır. Tablo gerçekten çok kalabalıksa yine yeniden hashing gerekebilir. Ama bu oran oldukça düşüktür: yarı yarıya dolu bir tabloda algoritmanın başarısız olma ihtimali neredeyse sıfırdır.

11.2 Arama

Komşuluk boyu H (hocanın "max length" dediği değer, deneme dizisinin boyu) ise arama, önce $h(x)$'in gösterdiği slota, sonra $h(x)+1$, $h(x)+2$, ..., $h(x)+H-1$ slotlarına bakar. Bunlar arandıktan sonra eleman ya bulunmuş ya da bulunamamıştır; daha fazla aramaya gerek yoktur. Arama karmaşıklığı $O(1)$ sayılır; performansı yüksektir.

11.3 Ekleme Örneği

Sembolik gösterimde (ekrandan, 02:10:00) a, b, c, d, r, s, t anahtarları tabloya yerleşmiştir. Her anahtarın altındaki sayı ev (base) slotudur:

Slot	0	1	2	3	4	5	6	7	8	9
Eleman		a (1)	b (1)	c (3)	d (4)	r (3)	s (5)	t (7)		

x eklenmek isteniyor; komşuluk boyu $H = 4$ seçilmiştir. Yani x ev slotundan en fazla 3 uzakta olmalıdır. x'in hash değeri 1'dir; 1-4 aralığında bir yere yerleşmelidir. Doğrusal denemeyle ilk boş yer 8'dir; ama 8 komşuluğun dışındadır, bu yüzden x'e bir yer açılmalıdır.

- **Adım 2(a):** Boşluk 8'dedir. Yer değiştirme adayları $8 - (H - 1) = 5$'ten başlayarak 5, 6, 7 numaralı slotlardır. 5'te r vardır; r'nin ev slotu 3'tür, 8'e taşınırsa komşuluk kuralı bozulur; r yerinden çıkarılamaz. Sonra 6 denenir: s'nin ev slotu 5'tir, 8'e taşınınca hâlâ komşuluğundadır. s ile boşluk yer değiştirir; boşluk 6'ya gelir.
- **Adım 2(b):** 6 hâlâ 1'den çok uzaktır. Adaylar $6 - (H - 1) = 3$'ten başlayarak 3, 4, 5'tir. c'nin ev slotu 3'tür ve 6, c'nin komşuluğundadır; c 6'ya taşınır, boşluk 3'e gelir.
- 3, x'in ilk dört gözü (1-4) içindedir; x 3'e yerleşir.

Slaytta, boşluk yerleşebileceği bir konuma gelene ya da artık taşınamayana kadar taşınacağı, taşınamazsa tablonun yeniden boyutlandırılması gerektiği yazar.

11.4 Silme

Silme için önce eleman bulunur; bu arama karmaşıklığıyla aynıdır. Sonra tembel silme (lazy deletion) yapılır: silinen elemanın yerine NULL ya da programda seçilen özel bir değer konur, böylece silinen yer işaretlenir. Bu basit bir silme işlemidir. Tembel silmede hep konu edilen risk burada da geçerlidir: çok fazla silme olursa arama zorlaşabilir.

11.5 Hop Bitmap Gösterimi

Sembolik gösterimde anahtarların altına ev slotu yazılmıştı; kodlarken bunu kontrol etmek için farklı yöntemler önerilmiştir. Bunlardan biri bitmap gösterimidir. Her slot için H bitlik bir dizi tutulur. Örneğin ev slotu 5 olan bir slotun dizisi 0100 ise: slotun kendisindeki eleman 5'e ait değildir, hemen yanındaki eleman 5'e aittir, diğer ikisi değildir. Bir, o slota ait komşunun hangisi olduğunu; sıfır, o komşunun başka bir slota ait olduğunu gösterir.

Örnek tabloda (ekrandan, 02:16:00) 3. göz için (hocanın anlatımıyla): kendi elemanı kendisindedir, ilk bit 1; yanındaki eleman 4'e aittir, bit 0; bir sonraki 3'e aittir, bit 1; diğeri başka bir slota aittir, bit 0.

11.6 Bitmap ile Ekleme Örneği

Tablo (ekrandan, 02:16:00); ev slotları A: 7, B: 9, C: 6, D: 7, E: 8, F: 12, G: 11, H: 9:

Slot	6	7	8	9	10	11	12	13	14
Eleman	C	A	D	B	E	G	F		
Hop	1000	1100	0010	1000	0000	1000	1000	0000	0000

H'nin eklenmesi. H normalde 9. slota yerleşmelidir. Doğrusal denemeyle ilk boş yer 13'tür; 13 çok uzaktır ($H = 4$ için 9-12 aralığı), H oraya konamaz. 13'ün komşuluğunda yer değiştirme adayları E, G ve F'dir (10, 11, 12).

- E'ye bakılır: E'nin bulunduğu 10. slotun hop dizisi 0000'dır, ilk biti 0'dır. Yani E 10. slota ait değildir, yukarıdan bir yerden (ev slotu 8) gelmiştir. Daha aşağı çekilirse erişilemez olur; E yerinden kıpırdatılamaz.
- G'ye bakılır: 11. slotun ilk biti 1'dir, G kendi slotundadır. G 13'e indirilebilir (11'den 2 uzak). G 13'e taşınır, boşluk 11'e gelir.

- 11, 9'a yeterince yakındır; H 11'e konur.

Sonuçta 9. slotun hop dizisi 1000'den 1010'a, 11. slotunki 1000'den 0010'a döner (G artık iki aşağıda, 13'te).

I'nın eklenmesi. Ardından ev slotu 6 olan I eklenmek istendi. Bu adımların karesi alınmadı; aşağıdaki anlatım altyazıdan kuruldu.

- İlk boş yer 14'tür. Adaylar 11, 12, 13'tekilerdir (H, F, G). 11. slotun ilk biti 0'dır; H kendi slotunda değildir, taşınamaz. Ama 11'in dizisi yol gösterir: 11'e ait bir eleman 13'tedir (G). G (ev 11) 14'e taşınabilir; boşluk 13'e gelir.
- 13, 6'dan hâlâ çok uzaktır. Yeni adaylar arasında E taşınamaz (kendi slotunda değil). F (12. slot, ev 12) kendi yerindedir; 13'e indirilir, boşluk 12'ye gelir.
- 12 hâlâ uzaktır. Adaylar B, E ve H'nin bulunduğu 9, 10, 11'dir. B (ev 9) kendi slotundadır; 12'ye indirilir, boşluk 9'a gelir.
- 9, I'nın ev slotu 6'nın 4'lük komşuluğu içindedir; I 9'a yerleşir.

Yerleştirmelerden sonra hop dizileri güncellenmelidir. Örneğin 6. slotun dördüncü komşusunda artık o slota ait bir eleman (I) olduğu için 6. slotun dizisi 1001'e çekilir.

12. Genişletilebilir Hashing (Extendible Hashing)

12.1 Motivasyon

Bu dersin son konusu genişletilebilir hashingdir. Slayttaki gerekçeler (ekrandan, 02:20:00):

- Veri ana belleğe sığamayacak kadar büyükse ne olur?
- Bu durumda asıl ölçüt, veriye ulaşmak için gereken disk erişimi sayısıdır.
- Hash fonksiyonunun da dinamik olarak değiştiği esnek bir yönteme ihtiyaç vardır.
- Deneme ya da ayrı zincirleme kullanılırsa, iyi dağıtılmış bir tabloda bile çakışmalar arama sırasında birkaç bloğun incelenmesine yol açabilir.
- Tablo çok dolunca da $O(N)$ disk erişimi gerektiren son derece pahalı bir yeniden hashing yapılmalıdır.

Genişletilebilir hashing, diğer yöntemlerin aksine çok sayıda ekleme ve silme yapıldığında bile yüksek performans veren bir yöntemdir ve işi az disk erişimiyle çözer.

12.2 Yapı

- **Kovalar (bucket):** Disklere, disklerin farklı bölümlerine yerleştirilir.
- **Dizin (directory):** Kovaları gösteren adresleri tutar; önek (prefix) bitleri üzerinden çalışır.

Örnekte (ekrandan, 02:25:35) iki bitlik önekler vardır: 00 öneki bir kovayı gösterir ve o kovadaki bütün elemanların ilk iki biti 00'dır. 01'in gösterdiği kovadaki elemanlar 01 ile, 10'unkiler 10 ile, 11'inkiler 11 ile başlar.

Veri 6 bitlik sayılardan oluşsun. Disk donanımının gereği olarak bir anda ulaşılabilecek eleman sayısı M ise her kovada en fazla M eleman olacak şekilde tasarım yapılır. Saklanacak kayıt sayısı zamanla değişebilir, artabilir; o zaman tabloda değişikliğe gitmek gerekir. Örnekte $M = 4$'tür. Yapı bir ağaca benzetilirse: kök dört işaretçi tutar, bu işaretçiler verinin ilk iki bitiyle ilişkilendirilmiştir; her yaprak en fazla dört eleman tutar ve bir yapraktaki bütün elemanların ilk iki biti aynıdır.

Başlangıç verisi:

Dizin	00	01	10	11
Kova	000100, 001000, 001010, 001011	010100, 011000	100000, 101000, 101100, 101110	111000, 111001

Slaytta her kovanın üstündeki parantezli sayı (örneğin (2)) kovanın kaç bitlik önekle ayırt edildiğini gösterir.

12.3 Ekleme

Ekleme iki tür değişikliğe yol açabilir: yapraklarda bir bölünmeye ya da kökte (dizinde) bir bölünmeye.

100100'ün eklenmesi. İlk iki biti 10 olduğu için 10 kovasına yerleşmelidir. Ama orada zaten dört eleman vardır ve kovanın en fazla eleman sayısı 4'tür; buraya yerleştirilemez. Kova bölünür ve dizin artık 2 bitle değil 3 bitle gösterilir: ilk üç biti 100 olanlar ve 101 olanlar ayrılır. Yeni elemanla birlikte beş eleman iki kovaya dağılır:

Dizin	000, 001	010, 011	100	101	110, 111
Kova	000100, 001000, 001010, 001011	010100, 011000	100000, 100100	101000, 101100, 101110	111000, 111001

Dizin 3 bite çıktı ama diğer kovaların adreslemesinde değişiklik olmadı: 000 ve 001 yine aynı kovayı, 010 ve 011 yine aynı kovayı gösterir. O kovalara hiç erişilmedi, yalnız dizin güncellendi. Ekleme basit bir şekilde yapılmış oldu.

000000'ın eklenmesi. Bütün bitleri sıfır olan bir değer eklenmek istendi. İlk üç biti 000 olduğu için 000/001'in gösterdiği kova sorumludur; ama orada da dört eleman vardır. Bu kovada da bölünme gerekir. Fakat dizinin önekleri bu ayrımı kaldıracak düzeydedir; 4 bite çıkmaya gerek yoktur. İlk üç biti 000 olanlar bir kovaya, 001 olanlar diğerine ayrılır: 000100 ile yeni gelen 000000 birinci kovaya, diğer elemanlar (001000, 001010, 001011) ikinci kovaya konur (ara adım eklendi). Yine yalnız o göze diskten ulaşılarak işlem yapıldı; diğerlerine ulaşmak gerekmedi.

12.4 Silme

Hoca silmeyi de anlattı: 100 ve 101 kovalarından bir eleman silinirse iki kovanın toplam eleman sayısı dörde iner ve birleştirilebilir hâle gelir. Bu durumda kovaların kendisine ulaşmaya gerek yoktur; iki dizin girdisi aynı kovayı gösterecek şekilde ayarlanarak kovalar birleştirilir.

12.5 Disk Erişimi

Kovaların boyutu çok küçük olursa yapı genişler, geniş bir hâl alır; genişlemek de iyi değildir. Bu durumda kovalarda değerlerin kendisi yerine işaretçiler tutularak erişmek mümkün olabilir; amaç disk erişimini olabildiğince azaltmaktır. Dizin belleğe sığmayacak kadar genişse ikinci bir disk erişimi gerekir; ama sonuçta işlemler en fazla iki disk erişimiyle yapılır.

12.6 Hocanın Kapanış Notu

Hoca, ispatlara girmesinin sebebinin öğrencilerin ileride bunları kalem kağıtla hesaplaması değil, yöntemlerin mantığının daha kolay oturması olduğunu söyledi. Formülasyon üzerinden gitmek işleyişi pekiştirir; özellikle akademik çalışma yapmak isteyenler için bu eksiklik ileride ortaya çıkar.

Örnek Sorular

Soru 1 (ekrandan, 00:10:00 ve 00:20:00). Yük faktörü $\alpha = n/m < 1$ olan açık adresli bir tabloda, düzgün hashing varsayımıyla başarısız aramadaki beklenen deneme sayısının en fazla $1/(1-\alpha)$ olduğunu gösterin. $\alpha = 0{,}5$ ve $\alpha = 0{,}9$ için sınır nedir?

> **Çözüm (derste verildi):** $\Pr\{X \geq i\} = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+2}{m-i+2} \leq \alpha^{i-1}$. $E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\} \leq \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}$. $\alpha = 0{,}5$ için 2, $\alpha = 0{,}9$ için 10 deneme.

Soru 2 (ekrandan, 00:23:40). $h(x) = x \bmod 7$ ve doğrusal denemeyle 13, 15, 6, 24 sırayla eklenir, ardından 23 eklenir. Tablo nasıl görünür ve yeniden hashing için yeni tablo boyutu ne seçilir?

Çözüm (derste verildi): 13 → 6, 15 → 1, 6 → 6 dolu, başa sarar → 0, 24 → 3, 23 → 2. 7 slotun 5'i dolu, tablo %70'in üstünde dolu. Yeni boyut, $2 \cdot 7 = 14$'ten büyük ilk asal olan 17; yeni fonksiyon $h(x) = x \bmod 17$.

Soru 3 (ekrandan, 00:32:35). $p = 17$, $m = 6$ iken $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$ ailesinden $h_{5,7}(3)$, $h_{3,4}(28)$ ve $h_{10,2}(3)$'ü hesaplayın.

Çözüm (derste verildi): $h_{5,7}(3) = 22 \bmod 17 = 5$, $5 \bmod 6 = 5$. $h_{3,4}(28) = 88 \bmod 17 = 3$, $3 \bmod 6 = 3$. $h_{10,2}(3) = 32 \bmod 17 = 15$, $15 \bmod 6 = 3$.

Soru 4 (ekrandan, 00:35:10; derste sözlü, 00:35:36). Satırları 1000, 0111, 1110 olan 3×4 matrisle $x = (1, 0, 1, 0)^T$ anahtarının hash değeri nedir? $b \times u$ boyutlu matrislerle kaç farklı hash fonksiyonu üretilebilir?

Çözüm (derste verildi): Mod 2 çarpım: $h(x) = (1, 1, 0)^T$. Matriste ub eleman, her biri 0 ya da 1: 2^{ub} fonksiyon. 32 bitlik anahtar ve $m = 256 = 2^8$ için 8×32 matris, 2^{256} fonksiyon.

Soru 5 (ekrandan, 01:06:45 ve 01:10:00). n anahtar ve m slot için en az bir çakışma olasılığını yazın. $n = 1000$, $m = 100000$ için bu olasılık $0{,}9932$ ise 100 rastgele fonksiyondan hiçbirinin çakışmasız olmama olasılığı nedir?

Çözüm (derste verildi): $\Pr(\#\text{çakışma} \geq 1) = 1 - \left(\frac{m-1}{m}\right)^{n(n-1)/2}$. 100 fonksiyonun hepsinde çakışma: $0{,}9932^{100} = 0{,}507$; en az birinin çakışmasız olma şansı yaklaşık %50.

Soru 6 (ekrandan, 01:19:05 ve 01:20:00). $K = \{8, 22, 36, 75, 61, 13, 84, 58\}$, $p = 87$, $a = 64$, $b = 5$, $m = 8$ ile mükemmel hashingde birinci seviye yerleşim nedir? 3. adım kontrolü sağlanır mı?

Çözüm (derste verildi): Slot 1: 61; slot 2: 8, 84; slot 4: 75; slot 5: 22; slot 6: 13; slot 7: 36, 58. $\sum n_j^2 = 0 + 1 + 4 + 0 + 1 + 1 + 1 + 4 = 12 < 16$, şart sağlanır. Alt tablolar: slot 2 için $m_j = 4$, $a_j = 82$, $b_j = 53$ (84 → 0, 8 → 1); slot 7 için $m_j = 4$, $a_j = 10$, $b_j = 54$ (58 → 1, 36 → 2); tek elemanlı slotlar için $m_j = 1$, $a_j = b_j = 0$.

Soru 7 (ekrandan, 01:36:25). $m = n$ slotlu bir tabloya n anahtar konursa $E\left[\sum_{j=0}^{m-1} n_j^2\right] < 2n$ olduğunu gösterin.

Çözüm (derste verildi): $a^2 = a + 2\binom{a}{2}$ ile $E[\sum n_j^2] = E[\sum n_j] + 2E[\sum \binom{n_j}{2}] = n + 2\binom{n}{2}\frac{1}{m} = n + 2 \cdot \frac{n-1}{2} = 2n - 1 < 2n$.

Soru 8 (ekrandan, 02:00:00). Cuckoo hashingde hash değerleri A: (0, 2), B: (0, 0), C: (1, 4), D: (1, 0), E: (3, 2), F: (3, 4) olan anahtarlar bu sırayla eklenir. Son tablo nedir? Ardından G: (1, 2) eklenirse ne olur?

Çözüm (derste verildi): Tablo 1: 0 → A, 1 → D, 3 → F; tablo 2: 0 → B, 2 → E, 4 → C. G eklenince D, B, A, E, F, C sırayla yer değiştirir; C, G'yi tablo 1'in 1. gözünden atar ve aynı elemanlar aynı slotlar arasında döner. Döngü oluştuğu için tablolar yeniden hashlenmelidir.

Soru 9 (ekrandan, 02:10:00). Hopscotch hashingde $H = 4$; slot 1-7'de a(1), b(1), c(3), d(4), r(3), s(5), t(7) vardır (parantez içi ev slotu). Ev slotu 1 olan x nasıl eklenir?

Çözüm (derste verildi): İlk boş slot 8, komşuluk (1-4) dışında. Adaylar 5, 6, 7: r (ev 3) 8'e taşınamaz; s (ev 5) 8'e taşınır, boşluk 6'ya gelir. Adaylar 3, 4, 5: c (ev 3) 6'ya taşınır, boşluk 3'e gelir. x 3'e yerleşir.

Soru 10 (ekrandan, 02:16:00). Hopscotch tablosunda (slot 6-12: C, A, D, B, E, G, F; ev slotları A: 7, B: 9, C: 6, D: 7, E: 8, F: 12, G: 11) ev slotu 9 olan H nasıl eklenir? Hop dizileri nasıl değişir?

Çözüm (derste verildi): İlk boş yer 13, çok uzak. E (10. slot, hop 0000, ev 8) taşınamaz. G (11. slot, hop ilk biti 1) 13'e taşınır, boşluk 11'e gelir. H 11'e yerleşir. 9. slotun hop dizisi 1010, 11. slotunki 0010 olur.

Soru 11 (ekrandan, 02:25:35). 2 bitlik dizinli genişletilebilir hashingde (kova kapasitesi 4) 10 kovası 100000, 101000, 101100, 101110 ile doludur. 100100 eklenince ne olur?

> **Çözüm (derste verildi):** Kova bölünür, dizin 3 bite çıkar. 100 kovası: 100000, 100100; 101 kovası: 101000, 101100, 101110. Diğer kovalara dokunulmaz; 000/001, 010/011 ve 110/111 girdileri yine aynı kovaları gösterir.

Dersin Toparlaması

Ders, açık adreslemenin analiziyle başladı: düzgün hashing varsayımıyla başarısız aramadaki beklenen deneme sayısı en fazla $1/(1-\alpha)$'dır ve tablo doldukça bu sayı hızla artar. Bunun bir çözümü olan yeniden hashing, tabloyu iki katından büyük ilk asal boyuta taşır. Ardından güvenlik kaygısıyla evrensel hashinge geçildi: hash fonksiyonu, iki farklı anahtarın çakışma olasılığını en fazla $1/m$ yapan bir aileden rastgele seçilir. Asal sayı yöntemi ($((ax+b) \bmod p) \bmod m$) ve matris yöntemi ($b \times u$ rastgele ikili matris, mod 2 çarpım) ile böyle aileler kuruldu; matris yönteminin çakışma olasılığının $1/2^b$ olduğu ispatlandı. n anahtarda en az bir çakışma olasılığının hesabı, büyük tablolarda bile çakışmanın neredeyse kaçınılmaz olduğunu gösterdi.

Dersin ikinci yarısında en kötü durumda $O(1)$ erişim veren yöntemler incelendi. Mükemmel hashing statik anahtar kümelerinde iki seviyeli tablo ve $m_j = n_j^2$ ile çalışır; toplar ve kutular teoremi ikinci seviyede çakışmasız bir fonksiyonun kolay bulunacağını, ikinci teorem de toplam alanın beklenen değerinin $2n$'den küçük kalacağını gösterir. Cuckoo hashing iki tablo ve iki fonksiyonla sorgu ve silmeyi $O(1)$'de yapar; döngü oluşursa yeniden hashing gerekir. Hopscotch hashing doğrusal denemenin donanım avantajını koruyup her elemanı ev slotunun komşuluğunda tutar. Son olarak, belleğe sığmayan veriler için disk erişimini en aza indiren genişletilebilir hashing görüldü: önek bitli bir dizin, kova bölünmesi ve gerektiğinde dizinin büyümesiyle işlemler en fazla iki disk erişiminde yapılır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.